

Curso 2025-26

L3-2: Unity3D Microjuego Asteroids

Fundamentos de Videojuegos

Jose María Barambones

DLSIIS - ETSI Informáticos

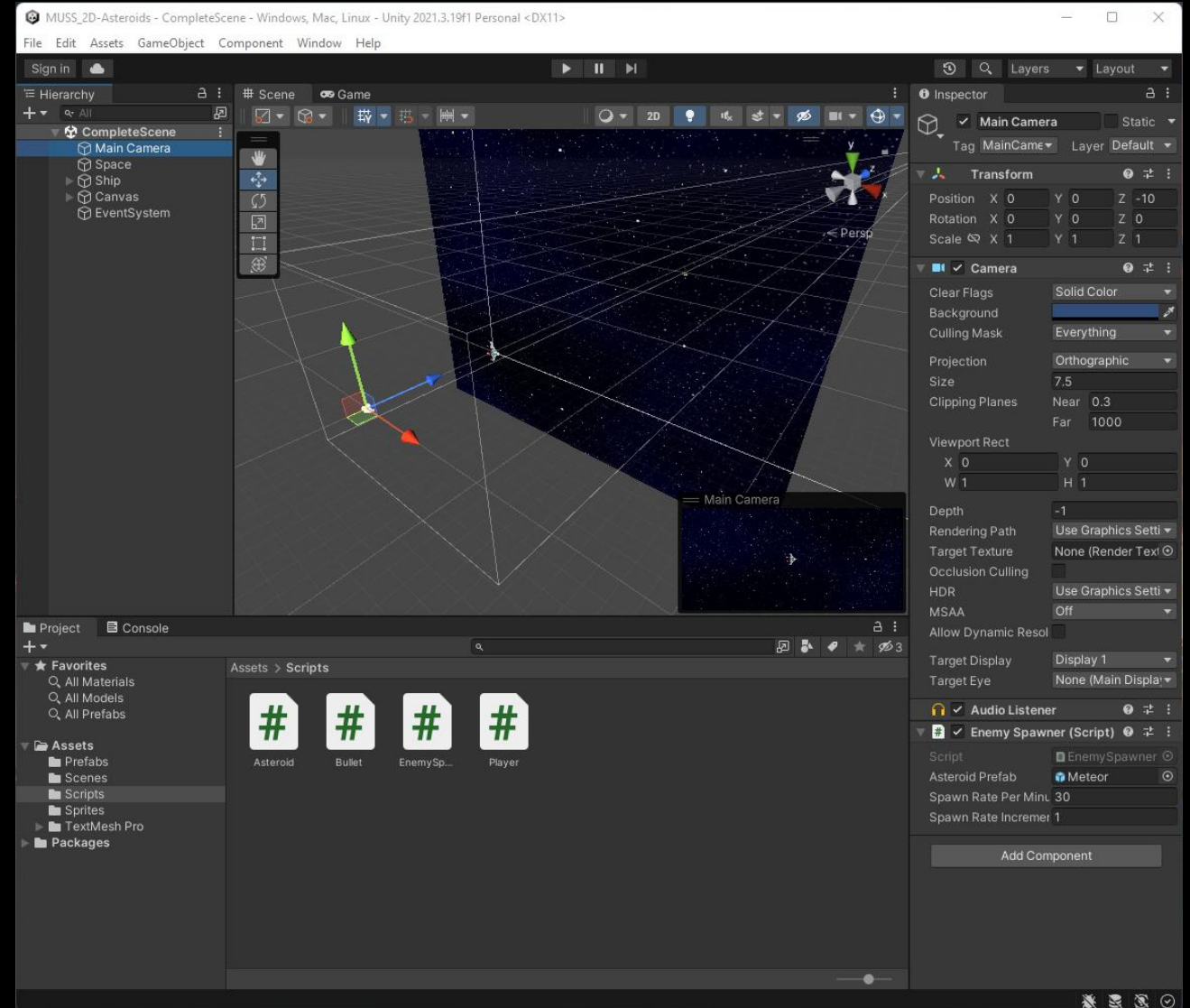
Universidad Politécnica de Madrid

L3-2: Unity3D - Microjuego Asteroids

- **Iteración 1: *Microdemo***
 - Escena 2D
 - Personajes
 - Mecánicas
 - Movimiento
 - Disparo
 - Enemy Spawn
 - UI: Scoring
- **Iteración 2: *Microjuego***
 - Mecánica extra: Fragmentos
 - UI: Pausa/Reinicio
 - Patrón de diseño: Pooling
- **Reglas de la entrega**

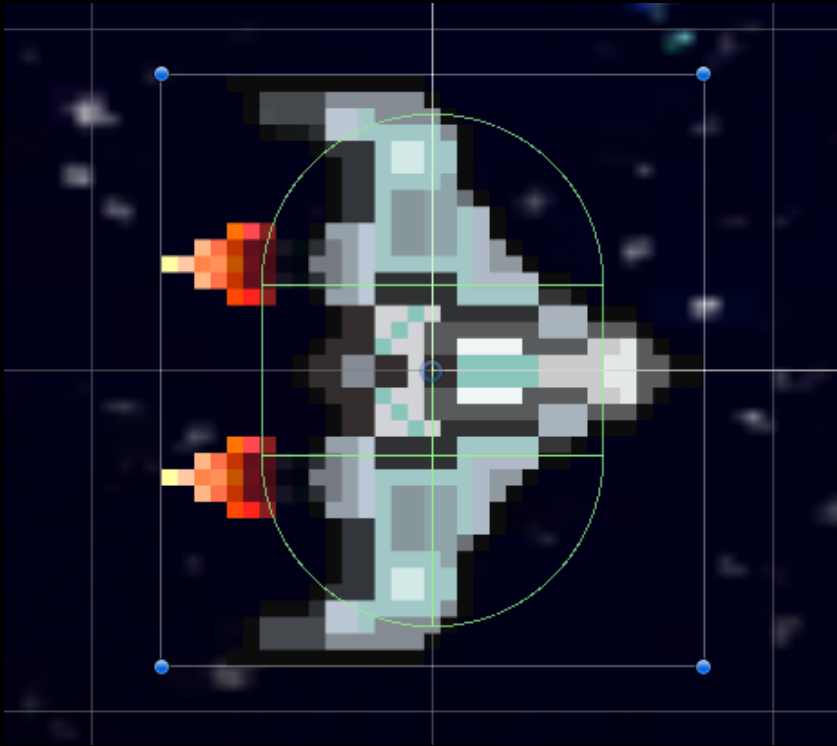
Iteración 1: Escena 2D

- Escena
 - Representa un nivel único con sus propios GameObjects.
 - 2 modos de escena:
 - Director (a través del editor).
 - Juego (a través de la cámara).
- Cámara
 - Proyección:
 - Perspectiva: **profundidad**
 - Ortográfica: **ortogonalidad**
 - Por convención en 2D:
 - Eje X → largo
 - Eje Y → alto
 - Eje Z → alzado
 - Propiedades:
 - *Field of view/Size*
 - *Clipping Planes (Near-Far)*

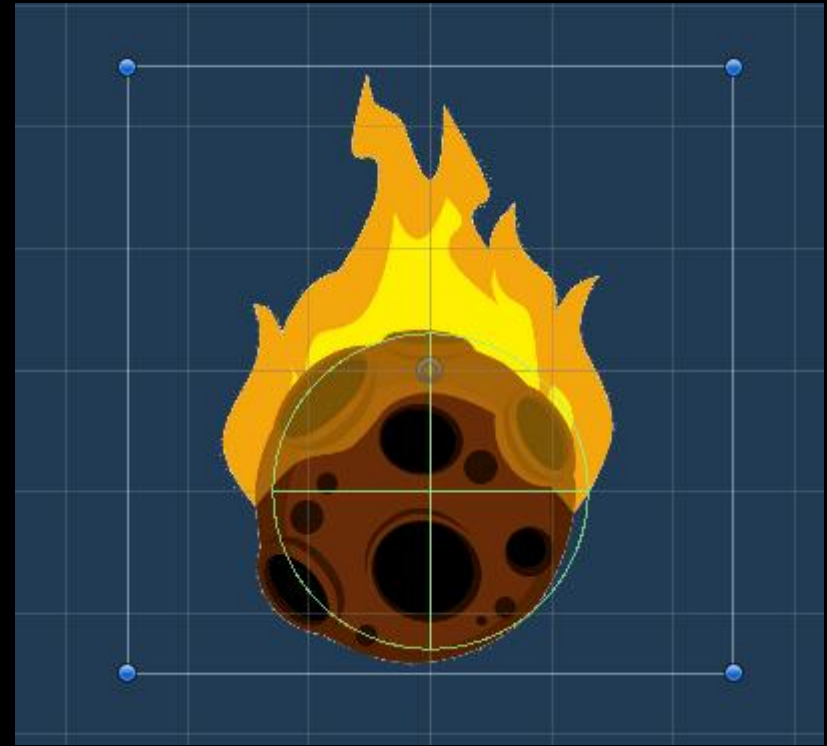


Iteración 1: Personajes

Ship

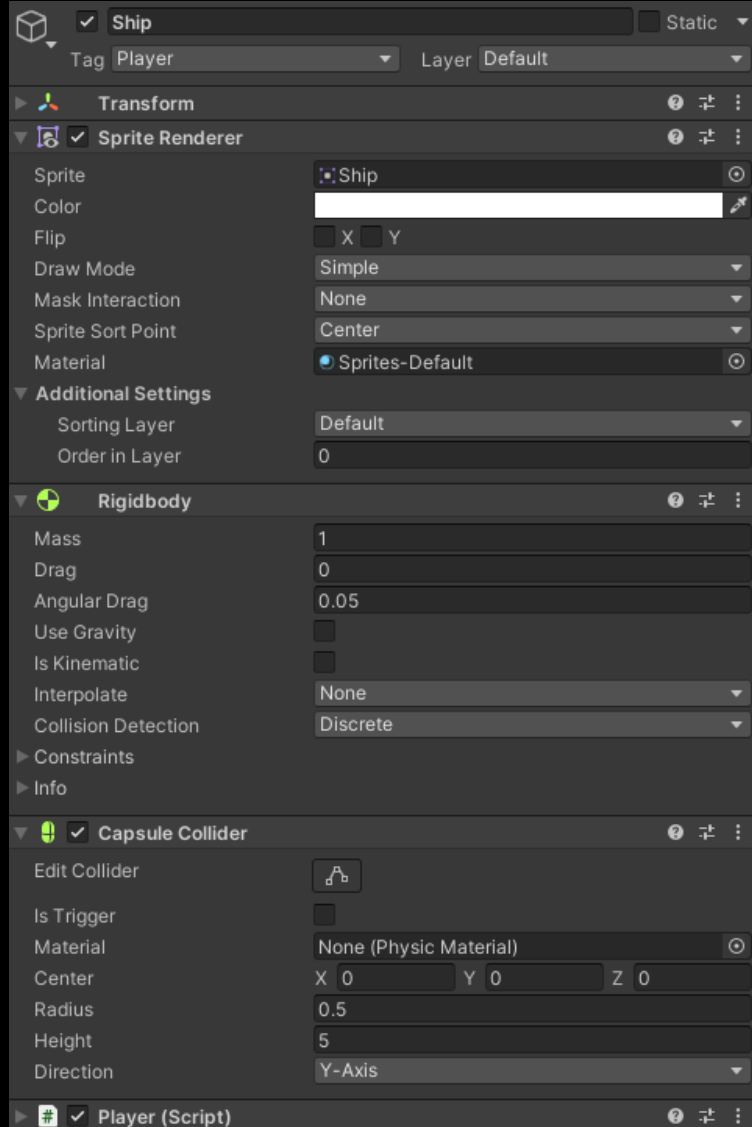


Asteroid (Prefab)

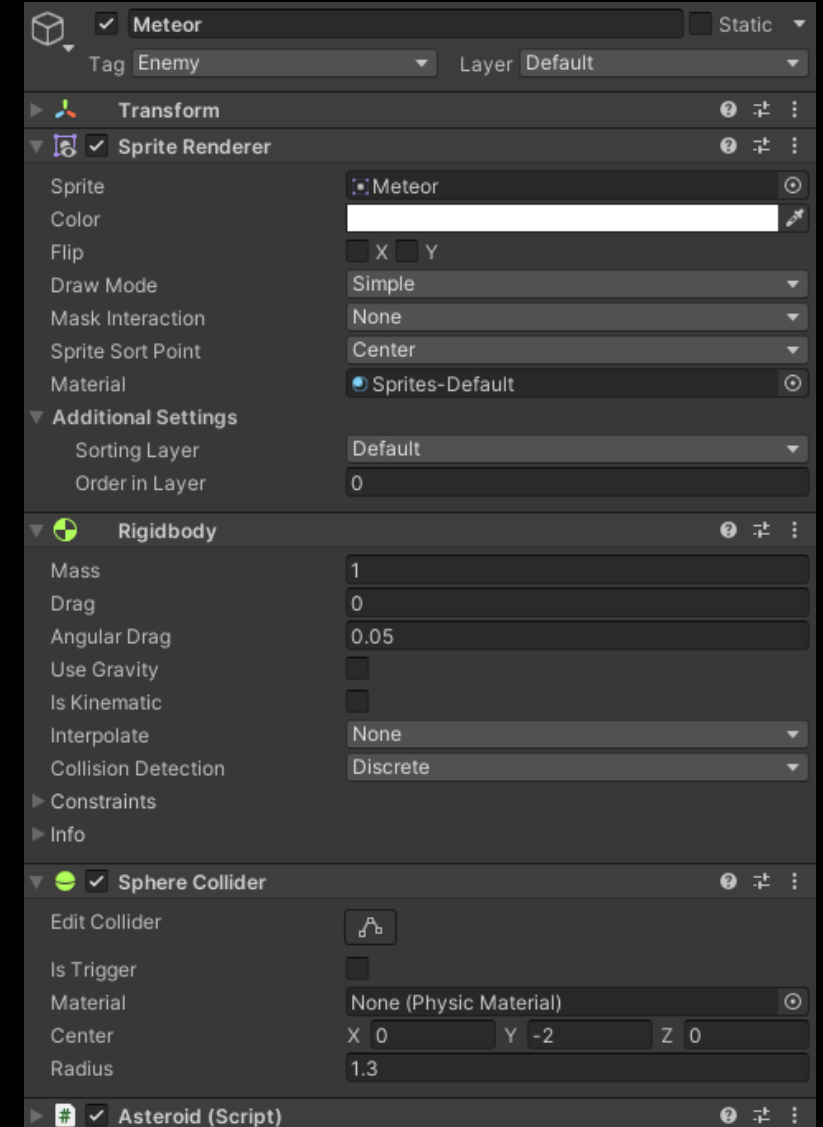


Iteración 1: Personajes

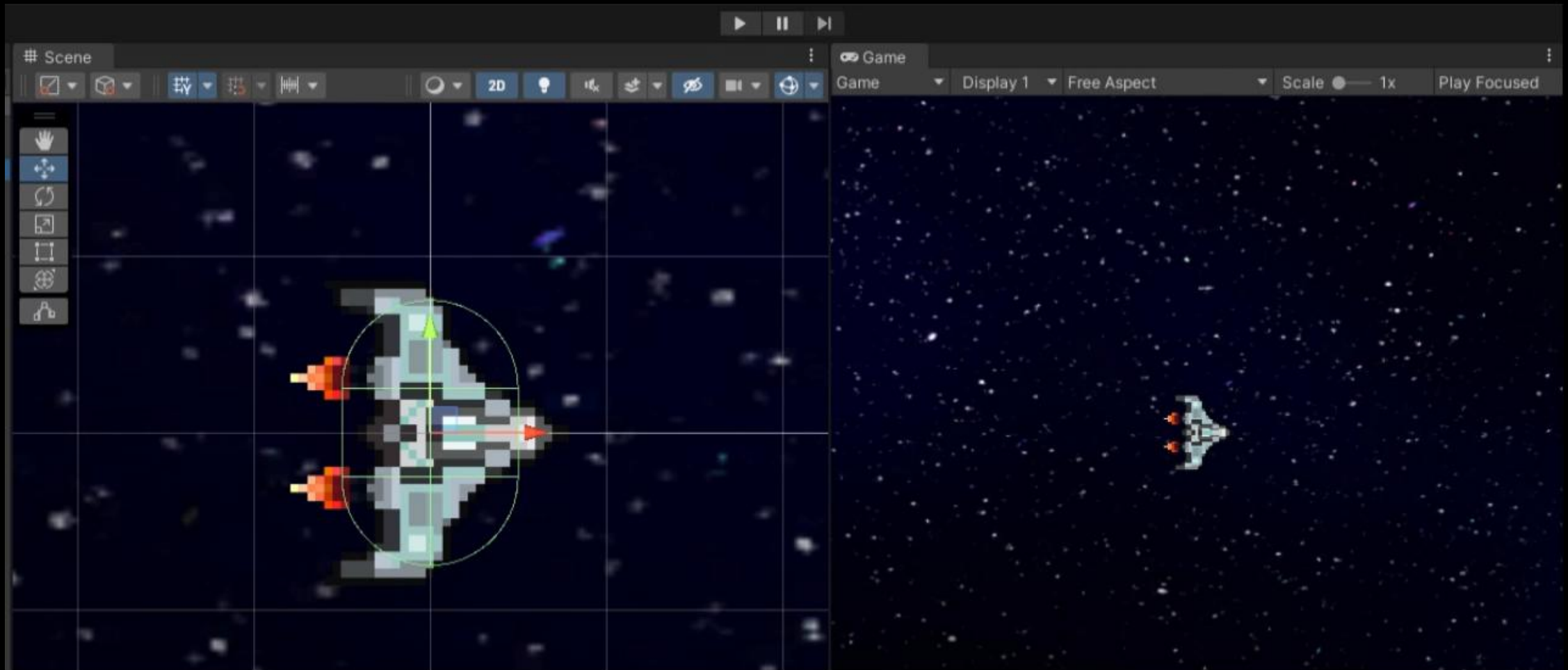
Ship



Asteroid (prefab)



Iteración 1: Mecánicas - Movimiento



Iteración 1: Mecánicas - Movimiento

- Desplazamiento con fuerzas
 - Calculado por el motor.
 - Mas kinestético (masa).
- Giro por *transform*
 - Calculado en el script.
 - Más control (sin masa).

```
public class Player : MonoBehaviour
{
    float thrustForce = 5f;
    float rotationSpeed = 120f;

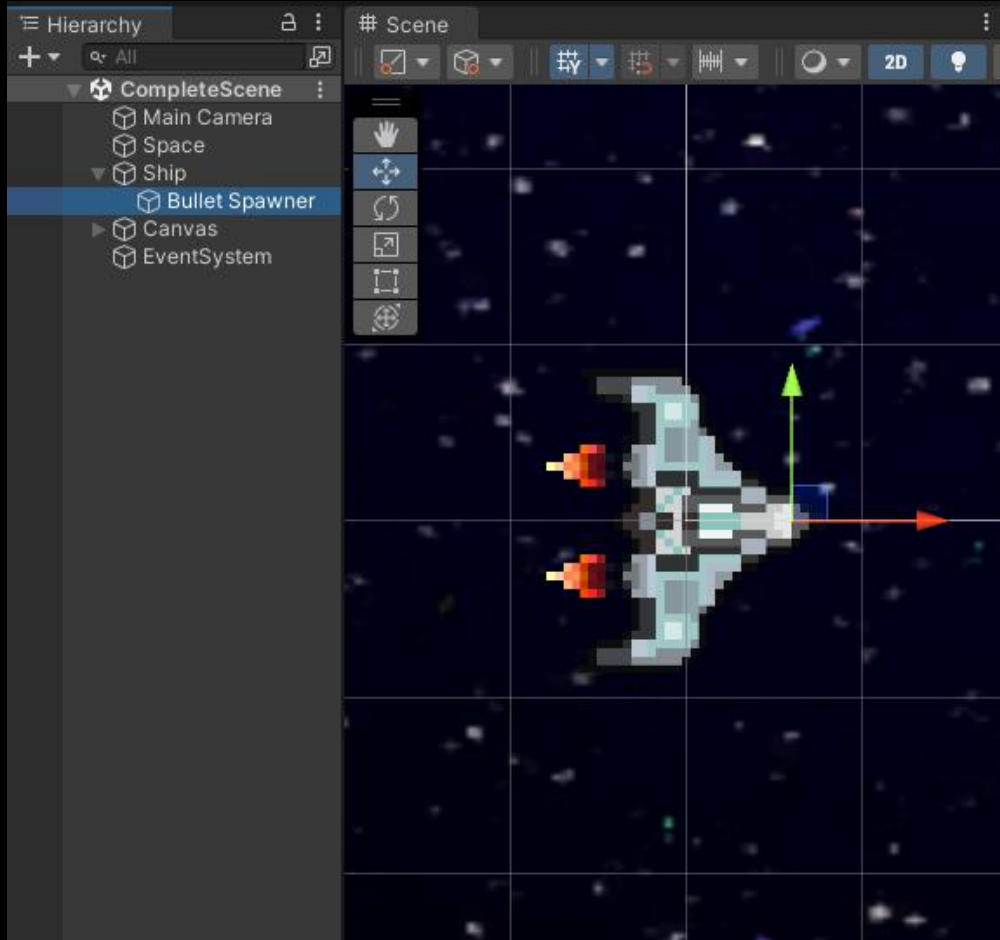
    Vector2 thrustDirection;
    Rigidbody _rigidbody;

    void Start()
    {
        // rigidbody nos permite aplicar fuerzas en el jugador
        _rigidbody = GetComponent<Rigidbody>();
    }

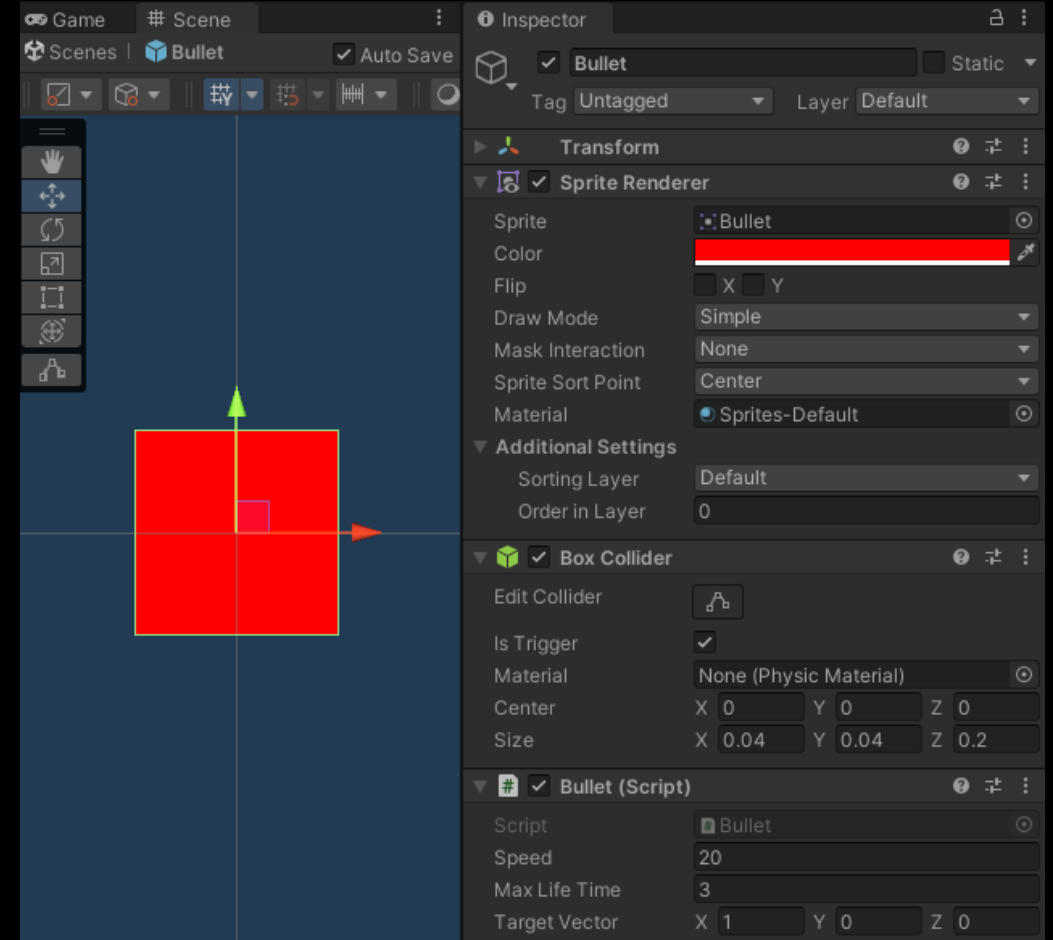
    private void FixedUpdate()
    {
        // obtenemos las pulsaciones de teclado
        float rotation = Input.GetAxis("Rotate") * rotationSpeed * Time.deltaTime;
        float thrust = Input.GetAxis("Thrust") * thrustForce;
        // la dirección de empuje por defecto es .right (el eje X positivo)
        thrustDirection = transform.right;
        // rotamos con el eje "Rotate" negativo para que la dirección sea correcta
        transform.Rotate(Vector3.forward, -rotation);
        // añadimos la fuerza capturada arriba a la nave del jugador
        _rigidbody.AddForce(thrust * thrustDirection);
    }
}
```

Iteración 1: Mecánicas - Disparo

Bullet Spawner (Gun)



Bullet (prefab)



Iteración 1: Mecánicas - Disparo

- Los GameObjects pueden anidarse para manipularse en relación a su objeto padre.
- Hay que prestar atención del transform del objeto padre
 - En nuestro caso desde el inspector (targetVector)
 - Vector3.right
- Si colisiona con un asteroide, ambos desaparecen.
- Si no colisiona, el objeto se mueve a la velocidad adecuada para desaparecer fuera de la escena
`Destroy(gameObject, maxLifeTime)`

```
public class Bullet : MonoBehaviour
{
    int speed = 10;
    float maxLifeTime = 3;
    public Vector3 targetVector;

    void Start()
    {
        // nada más nacer, le damos unos segundos de vida,
        // lo suficiente para salir de la pantalla
        Destroy(gameObject, maxLifeTime);
    }

    void Update()
    {
        // la bala se mueve en la dirección del jugador al disparar
        transform.Translate(targetVector * speed * Time.deltaTime);
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.tag == "Enemy")
            Destroy(other.gameObject);
    }
}
```

Iteración 1: Mecánicas - Vectores

- **Vector3** permite representar vectores y puntos en la escena
- Operadores directos:
 - “+” y “-” → suma de vectores
 - “*” y “/” con número → multiplicación escalar
 - “==” → “equals” con margen de error de coma flotante
- Propiedades:
 - Magnitud, normal, y componentes (x, y, z).
 - Vector3.forward → Vector3(0, 0, 1)
 - Vector3.up → Vector3(0, 1, 0)
 - Vector3.right → Vector3(1, 0, 0)
 - Vector3.one → Vector3(1, 1, 1)
 - Vector3.zero → Vector3(0, 0, 0)
 - ...
- **Vector2** utiliza las componentes x e y (“z” es 0) pero pueden combinarse.

Iteración 1: Mecánicas - Spawn

1. Instanciamos un asteroide fuera de la escena.
 - Aleatorio en la componente x.
 - Fijo en la componente y.
 - Al instanciar le damos un temporizador de “muerte”.
2. El objeto se desplaza automáticamente.
3. En otro script “Asteroid” (parecido a las balas):
 - Si colisiona con:
 - Bala → ambos desaparecen.
 - Nave → reinicia la escena.
 - Si no colisiona:
 - El objeto se mueve a la velocidad adecuada para desaparecer fuera de la escena

Destroy(gameObject, maxLifeTime)

```
public class EnemySpawner : MonoBehaviour
{
    GameObject asteroidPrefab;
    float spawnRatePerMinute = 30;
    float spawnRateIncrement = 1;
    float xBorderLimit, yBorderLimit;

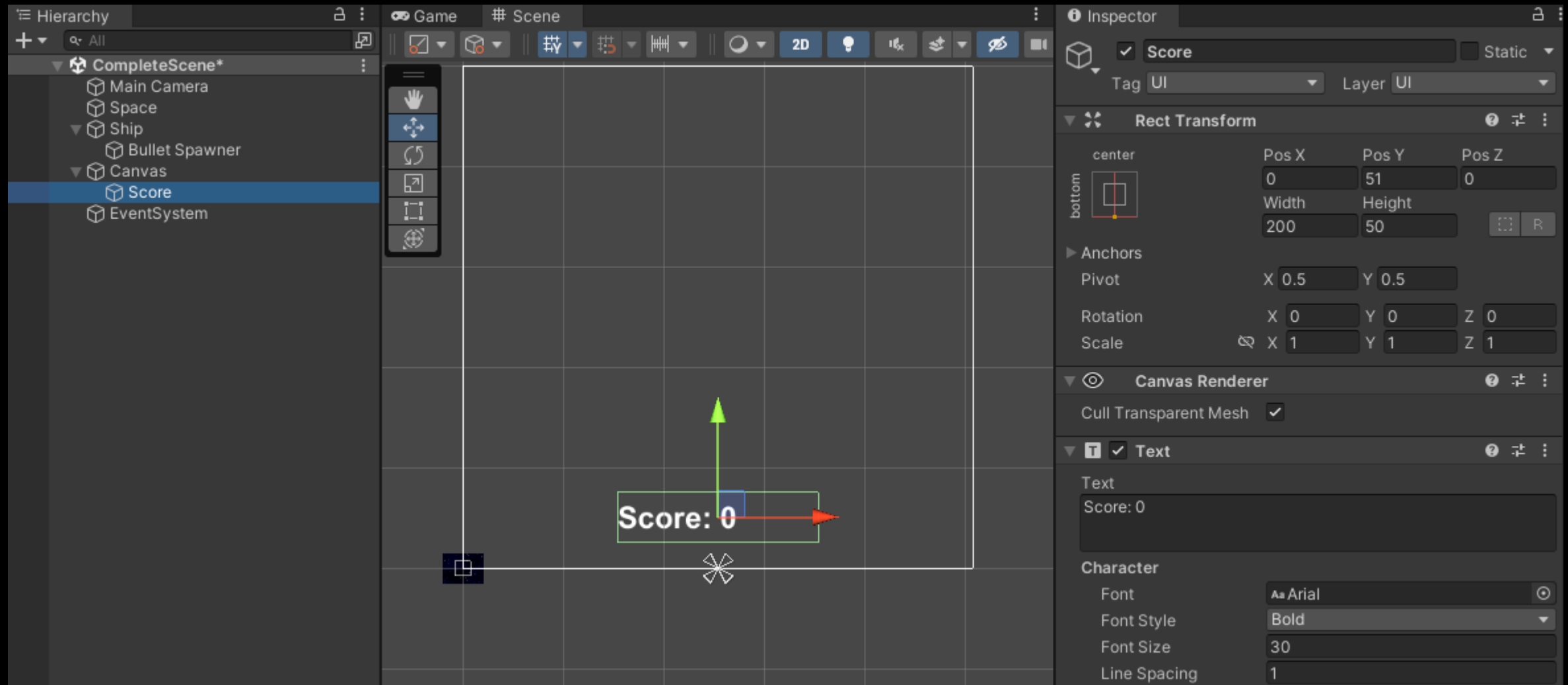
    private float spawnNext = 0;

    void Update()
    {
        // instanciamos enemigos sólo si ha pasado tiempo suficiente desde el último.
        if (Time.time > spawnNext)
        {
            // indicamos cuándo podremos volver a instanciar otro enemigo
            spawnNext = Time.time + 60 / spawnRatePerMinute;
            // con cada spawn hay mas asteroides por minuto para incrementar la dificultad
            spawnRatePerMinute += spawnRateIncrement;

            // guardamos un punto aleatorio entre las esquinas superiores de la pantalla
            var rand = Random.Range(-xBorderLimit, xBorderLimit);
            var spawnPosition = new Vector2(rand, yBorderLimit);

            // instanciamos el asteroide en el punto y con el ángulo aleatorios
            Instantiate(asteroidPrefab, spawnPosition, Quaternion.identity);
        }
    }
}
```

Iteración 1: IU - Scoring



Iteración 1: IU - Scoring

1. Creamos el *Canvas* con el elemento de texto (*Text*) en la interfaz.
2. Con cada colisión a un asteroide, incrementamos los puntos (dato estático).
 - Buscamos el *gameObject* con el texto a través de su tag (o nombre).

FindGameObjectWithTag()

Find()

- Acedemos a su componente *Texto*

GetComponent<nombre>()

- Editamos el componente con la nueva información.

```
public class Bullet : MonoBehaviour
{
    ...

    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.tag == "Enemy")
        {
            Destroy(other.gameObject);
            IncreaseScore();
        }
    }

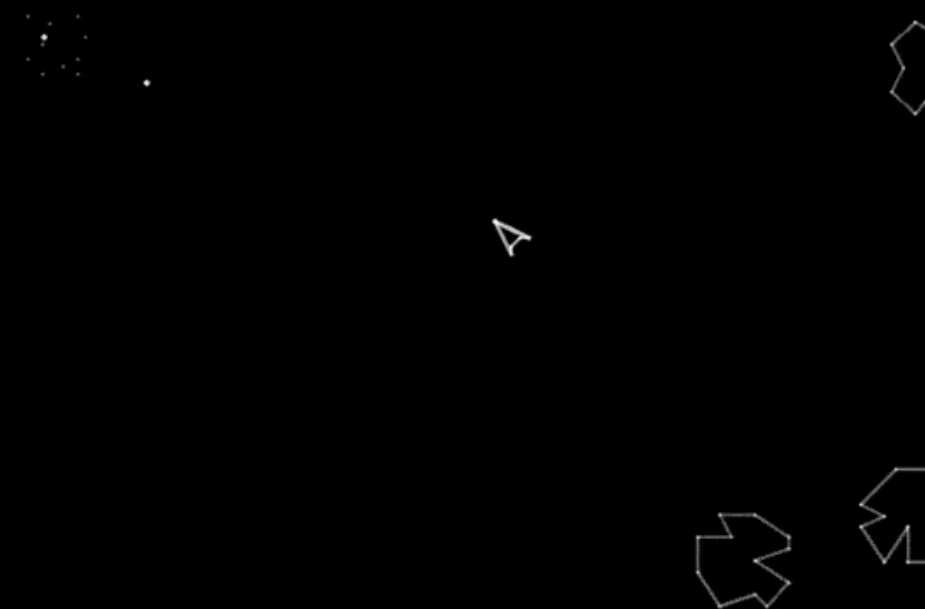
    public void IncreaseScore()
    {
        // cuando un asteroide es destruido, llama a esta función para darnos puntos.
        Player.SCORE++;
        UpdateScoreText();
    }

    private void UpdateScoreText()
    {
        // llamamos a esta función cada vez que ganamos puntos para actualizar el marcador
        GameObject go = GameObject.FindGameObjectWithTag("UI");
        go.GetComponent<Text>().text = "Score: " + Player.SCORE;
    }
}
```

Iteración 2

Mecánica extra: Fragmentos

- Con cada colisión asteroide-bala:
 - Surgen 2 mini-asteroides.
 - Se separan con un ángulo fijo siendo el vector de la bala la bisectriz.
 - El mini-asteroide desaparece si vuelve a colisionar con una bala

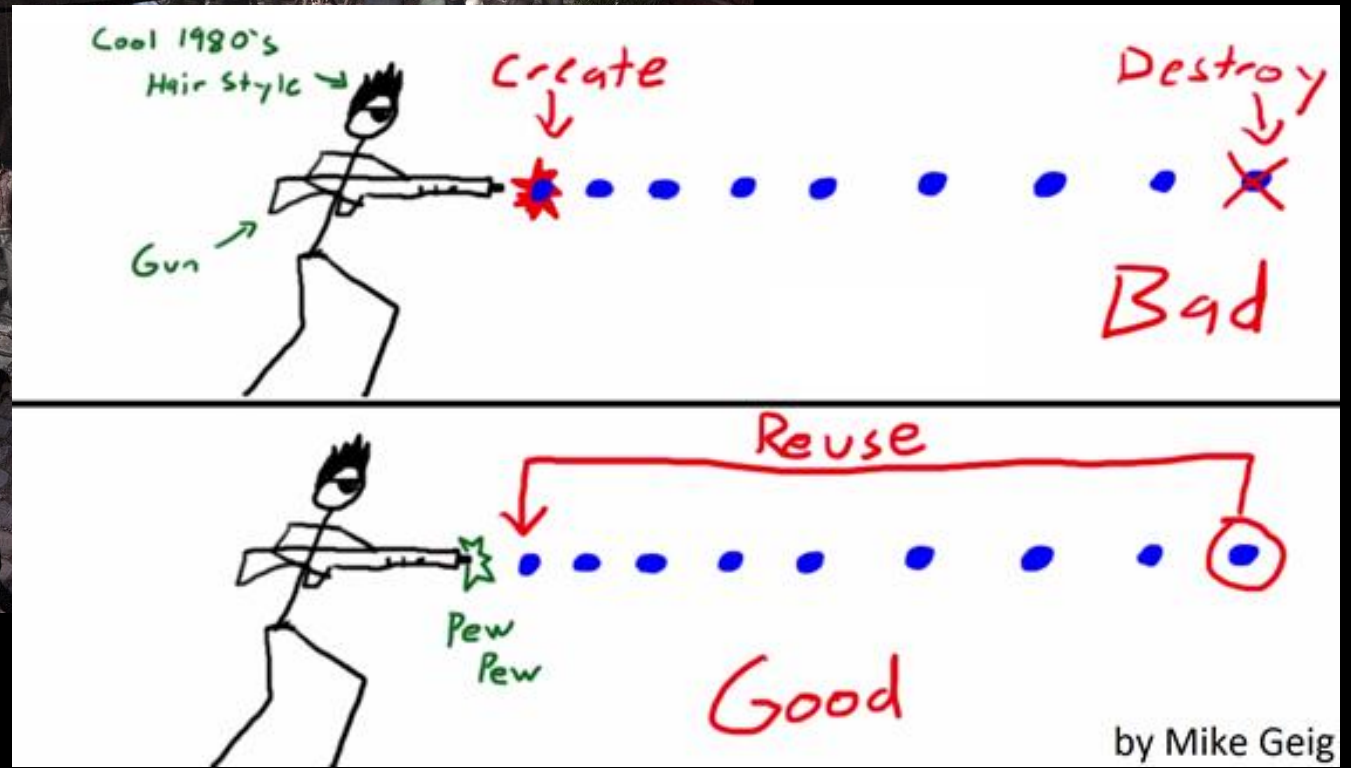


UI: Pausa/Reinicio

- Poder pausar el juego.
- Al pausar, la escena se para y no se puede interactuar con ella, salvo para reanudarla.
- 2 opciones para pausar/reanudar:
 - Presionando escape
 - Botón en la interfaz

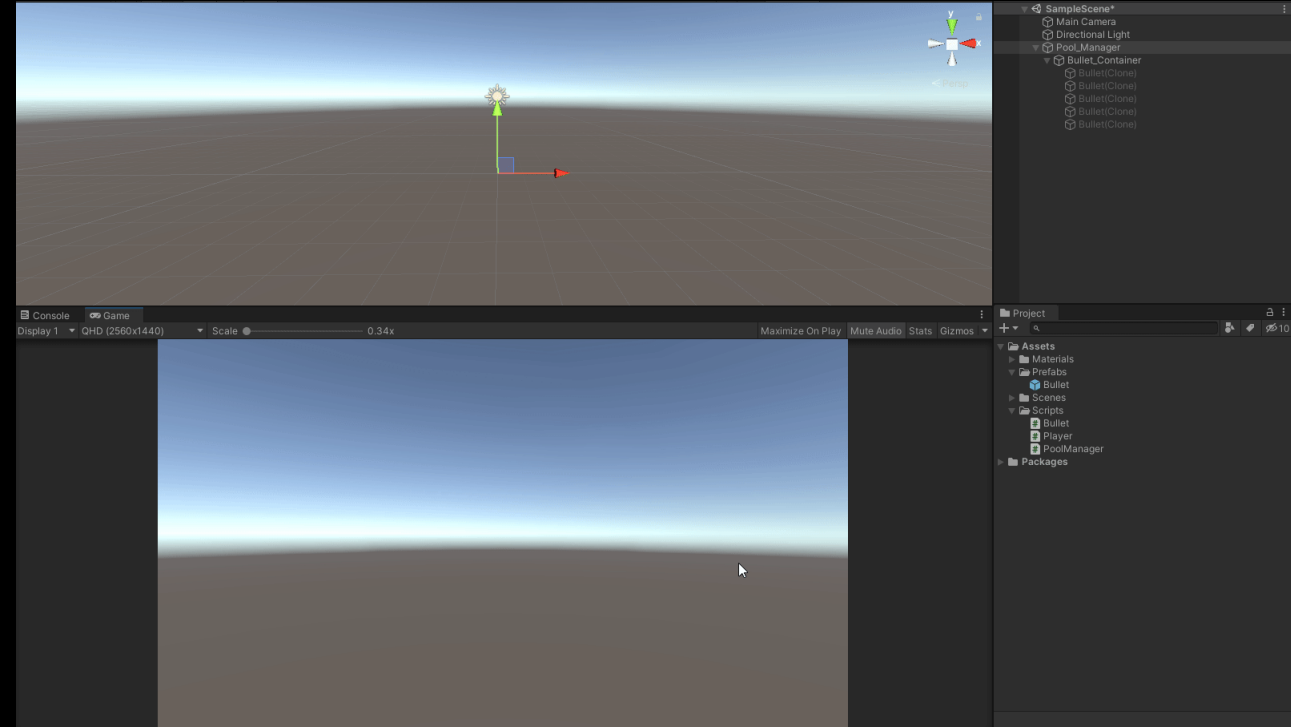


Iteración 2: Pooling



Iteración 2: Pooling

- La “Piscina de objetos” (“*Object Pooling*”) es un patrón de diseño software que usa un conjunto de objetos inicializados preparados para su uso.
- Esto es más efectivo normalmente que creando y destruyendo los objetos bajo demanda.
- Pasos (en Unity):
 1. Se inicializa la “pool” en un lugar de la escena no visible por la cámara.
 2. Se instancian un conjunto de objetos dentro del pool.
 3. El cliente del pool (“manager”) le pide un objeto para colocarlo en la escena.
 4. Cuando el objeto termina, se retorna al pool para hasta que se vuelva a necesitar.
- Este patrón de diseño sirve para mejorar significativamente el rendimiento evitando instanciar/destruir durante la ejecución de la escena.



Reglas de la entrega

- **Entregable individual/parejas (1 punto).**
 - Iteración 1: 60%
 - Iteración 2: 40%
- **Para cada entrega:**
 - Una demo jugable (fichero .exe, carpeta de datos y motor .dll).
 - Enlace al repositorio Git con el código fuente del proyecto Unity3D.
 - Documentación (mini GDD máximo 4 páginas)
 - Escenas
 - Assets y GameObjects
 - Mecánicas
 - Scripts
 - Interacción

Referencias

- <https://docs.unity3d.com/es/2021.1/Manual/CamerasOverview.html>
- <https://docs.unity3d.com/2023.2/Documentation/ScriptReference/Vector3.html>
- https://en.wikipedia.org/wiki/Object_pool_pattern
- <https://learn.unity.com/tutorial/introduction-to-object-pooling>
- <https://gamedevbeginner.com/the-right-way-to-pause-the-game-in-unity/>
- <https://vionixstudio.com/2020/11/26/pause-a-game-in-unity/>

Curso 2025-26

L3-2: Unity3D Microjuego Asteroids (fin)

Fundamentos de Videojuegos

Jose María Barambones

DLSIS - ETSI Informáticos

Universidad Politécnica de Madrid